RMS Instruments Communication via the VFBUS SL Protocol

Document identification: VF Q-7P1-10

Document number: VF2206060048

Printed version of the document is intended for information only.
Check the controlled digital version in the KARAT information system to verify validity.

	Position	Name
Prepared by:	VF's team of authors	

Revision number: B

Effective from: 2 June 2025

	Position	Name	Date	Signature
Approved by:	chairman of the Board of Directors and CEO of VF, a.s.	Jiří Malysák	23 June 2025	*Electronically signed*

Revision history

Revision	Date	Revision purpose/description
B	2 June 2025	Update according to the basic document Q-7P1-03 in revision H

Table of contents

1 General ... 4
1.1 Purpose and scope ... 4
1.2 Abbreviations ... 4
1.3 Terms and definitions ... 4
2 Introduction ... 5
3 Data space ... 6
3.1 Data space definition ... 6
3.2 Basic data types ... 7
3.2.1 Integers ... 7
3.2.2 Real time ... 8
3.2.3 Real numbers ... 8
3.2.4 Text strings ... 8
3.3 Expanded data types ... 9
3.3.1 Integers ... 9
3.3.2 Real numbers ... 10
3.3.3 UTF-8 text strings ... 10
3.4 Instrument identification ... 12
3.5 Main command, entry data buffer ... 13
4 Description of the VFBUS protocol ... 14
4.1 VFBUS transactions ... 14
4.2 Addressing ... 17
4.3 VFBUS SL protocol ... 18
4.4 Checksum calculation ... 19
5 Example of communication ... 22
5.1 VFBUS transactions ... 22
5.1.1 VFBUS SL protocol ... 22
6 Documents Used ... 23
7 Annexes ... 23
8 Final provisions ... 23

List of figures

Fig. 1 Basic overview of protocols ... 5
Fig. 2 Data space and its areas ... 6
Fig. 3 Message structure ... 14
Fig. 4 VFBUS SL protocol message ... 18
Fig. 5 CRC checksum calculation ... 19
Fig. 6 CRC checksum calculation using a table ... 20

List of tables

Tab. 1 Basic distribution of the RO address space ... 7
Tab. 2 Basic distribution of the RW address space ... 7
Tab. 3 Basic integer data types ... 7
Tab. 4 Real time ... 8
Tab. 5 Basic real data types ... 8
Tab. 6 Text strings ... 8
Tab. 7 Codes of ASCII characters ... 9

Tab. 8 Expanded integer data types 9
Tab. 9 Expanded real data types 10
Tab. 10 UTF-8 encoding 10
Tab. 11 Text strings with UTF-8 characters 11
Tab. 12 Selected UTF-8 character codes 11
Tab. 13 DEV_ID structure - instrument identification, RO area, length 0x0024 12
Tab. 14 Encoding of identifiers HW_ID, SW_ID, PL_ID 13
Tab. 15 Structure of VF PDU 14
Tab. 16 Direct data transmission 15
Tab. 17 Read procedure 15
Tab. 18 Write procedure 16
Tab. 28 Addressing in devices 17
Tab. 19 Setting of an asynchronous serial line for the VFS protocol 18

1 General

1.1 Purpose and scope

This document describes the internal communication protocol of VF BUS SL modules and instruments of VF, a.s. on full-duplex serial lines. It is an excerpt of the necessary parts of the basic document communication of the device RMS Q-7P1-03. It is primarily intended for implementing communication with measuring devices and local evaluation units.

1.2 Abbreviations

ADU	Application Data Unit - protocol data framework
ASCII	American Standard Code for Information Interchange - table of character codes
CRC	Checksum (cyclic redundancy check)
HW	Hardware
OSI	Open Systems Interconnection, communication network model
PDU	Protocol Data Unit
RO	Read Only register or area of registers
RW	Read / Write register or area of registers
RWP	RW area of registers with conditional registration
RWU	RW area of registers excl. RWP registers
SW	Software
VFBUS	Protocol for instruments of VF, a.s.
VFBUS CAN	Implementation of the VFBUS protocol for CAN
VFBUS SL	Implementation of the VBUS protocol for serial lines
VFBUS UDP	Implementation of the VFBUS protocol for UDP
WO	Write only

1.3 Terms and definitions

Dedicated	If existing, it is addressed in a specific manner in a specific position. If not existing, the position **can be used** for something else.
Message	Transaction data transferred in one direction. One transaction consists of one message (direct transfer source -> target) or two messages (query + response).
Obligatory	Always existing, addressed in a specific manner in a specific position.
Packet	Auxiliary or data fragment of a message. One message has one or more packets. Auxiliary packets can be transferred in the direction opposite to the message data.
Position	Address space, register, byte, bit.
Reserved	If existing, it is addressed in a specific manner in a specific position. If not existing, the position **cannot be used** for anything else.
Transaction	One complete data transfer between two and more modules (processors).

2 Introduction

Standardization of communication protocols is a necessary prerequisite for mutual compatibility of products and modules. The MODBUS protocol is the worldwide communication standard used in radiation technology instruments. The MODBUS is a protocol of application layer messages that is located at level 7 of the OSI model. It provides client / server communication between instruments connected in various types of buses or networks.

The basic communication protocol to connect the company's products with the surrounding world through RMS networks, such as environment monitors, contamination alarms, etc., is the MODBUS RTU protocol on serial lines and the MODBUS on TCP protocol on the Ethernet. For the purposes of this document, these MODBUS protocols are identified as external.

Most of the company's products do not contain just one module or processor, they usually contain multiple modules. Internal networks are created within a single instrument, featuring multi-master communication. Values are rarely queried; they are transferred at the moment of their creation. The MODBUS protocol is unsuitable for this communication method. The VFBUS protocols are defined for internal communication. They include the VFBUS SL (VFS) protocol for serial lines, the VFBUS CAN (VFC) protocol for the CAN bus and the VFBUS UDP (VFU) protocol for Ethernet transfers via the UDP.

The link between internal and external protocols is a unified data space that complies with MODBUS specifications.

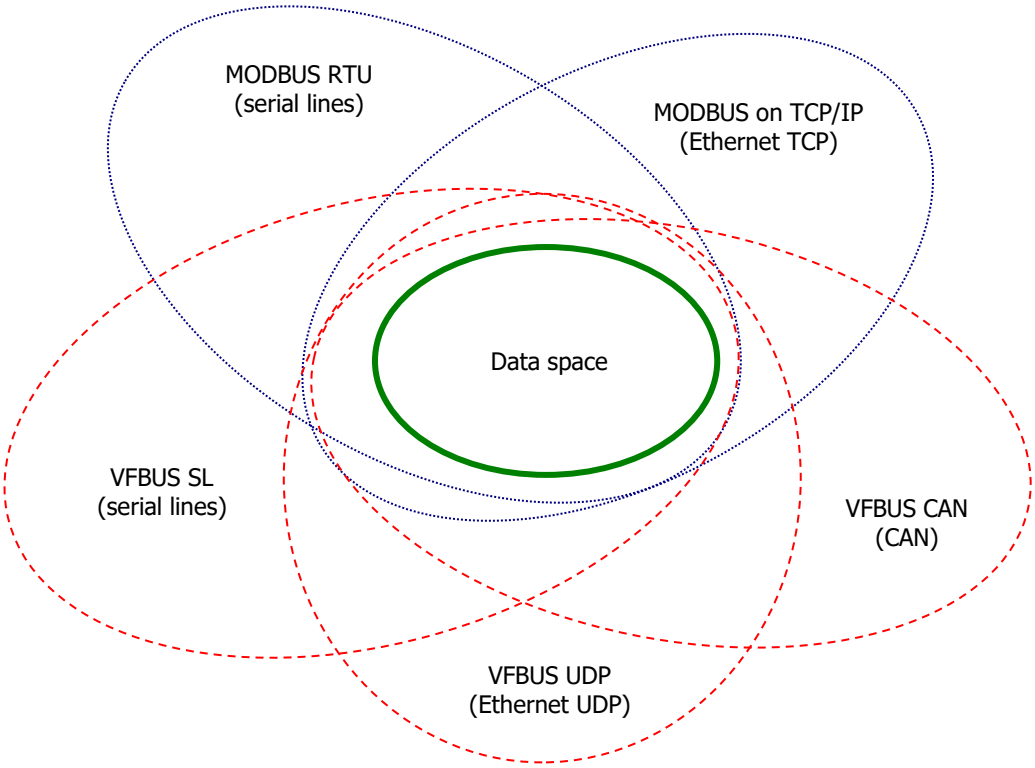

Fig. 1 Basic overview of protocols

3 Data space

3.1 Data space definition

All transferred information is stored in a uniform data space. The data space is composed of two separated address spaces. The **RO address space** contains read-only registers. The **RW address space** contains registers for reading or for reading and writing. Each space contains 16b registers provided with a 16b address. Each of the spaces contains a maximum of 65,536 registers and does not need to be continuous. In total, 256 kB of data can be addressed, and half of them are strictly read-only. Paging can further enhance the capacity.

Unless specified otherwise, the data format is little endian. Data with an odd number of bytes is always complemented with one upper byte with the value of 0 to ensure an even number. The mapping of basic data types into individual registers is described in Chapter 3.2. The mapping of expanded data types is described in Chapter 3.3. The RO space is obligatory and has the instrument identification described in Chapter 3.4 at its beginning. The RW space is not obligatory. It contains the main command described in 3.5.

The RW address space can have a sub-area with controlled access, marked as the **protected RW area (RWP)**. Key parameters that cannot be changed under normal operation are saved in this area. Writing is blocked in this area during normal operation, so it behaves as an RO area. To permit writing, it is necessary to use the main command or other HW devices (key). If the address space contains an RWP area, it is appropriate to describe RW registers, excluding the RWP area, as registers **of a unprotected RW area (RWU)** in the instrument documentation.

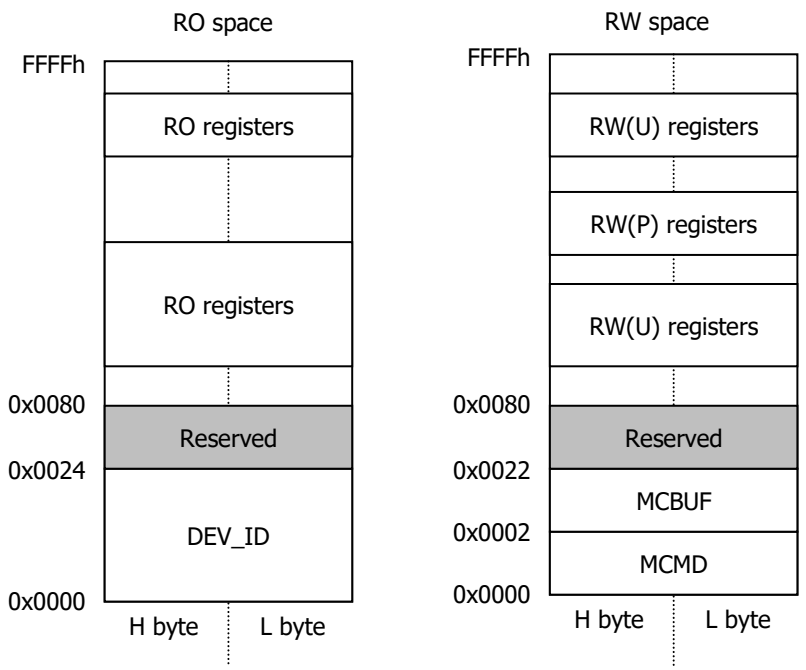

Fig. 2 Data space and its areas

Output data of the instrument, that are strictly RO, are placed in the RO address space. Input data or set-up parameters are placed in the RW address space. A parameter may also be placed in the RW address space if it is RO (or WO) in nature, however, its external overwriting (or reading) must not affect the instrument's functioning in any way.

A place in space that is not obligatory, reserved or dedicated is free for use.

Use	Start address	Number of registers	Designation	Type	Description
Obligatory	0x0000	0x0024	DEV_ID	DEV_ID	Structure of instrument identification, see3.4.
Reserved	0x0024	0x005C			Reserved for future extension.

Tab. 1 Basic distribution of the RO address space

Use	Start address	Number of registers	Designation	Type	Description
Reserved	0x0000	0x0002	MCMD	uint32	Main command of the instrument, see3.5.
Reserved	0x0002	0x0020	MCBUF	union	Input buffer of the main instrument, see3.5.
Reserved	0x0022	0x005E			Reserved for future extension.

Tab. 2 Basic distribution of the RW address space

3.2 Basic data types

3.2.1 Integers

Type	Length	Range	Description	Notation in registers		
				Symbolic hex notation	Register offset	Register content
uint8	8 bit	0 … 255	unsigned char	0x12	0	0x0012
int8	8 bit	-128 … 127	signed char, binary supplement, numbers 0 … 127 conforming 0x00 … 0x7F, numbers -128 … -1 conforming 0x80 … 0xFF	0x12	0	0x0012
uint16	16 bit	0 … 65535	unsigned short	0x1234	0	0x1234
int16	16 bit	-32768 … 32767	signed short, binary supplement, numbers -32768 … -1 conforming 0x8000 … 0xFFFF, numbers 0 … 32767 conforming 0x00 … 0x7FFF	0x1234	0	0x1234
uint32	32 bit	0 … $(2^{32} – 1)$	unsigned long	0x1234 5678	0	0x5678
					1	0x1234
int132	32 bit	-2^{31} … $(2^{31}-1)$	signed long, binary supplement, numbers -2^{31}… -1 conforming 0x8000 0000 … 0xFFFF FFFF, numbers 0 … $(2^{31}-1)$ conforming 0x0000 0000 … 0x7FFF FFFF	0x1234 5678	0	0x5678
					1	0x1234

Tab. 3 Basic integer data types

3.2.2 Real time

Type	Length	Range	Description	Notation in registers		
				Symbolic hex notation	Register offset	Register content
time	32 bit	0 … (2^{32} – 1)	number of seconds from UTC 00:00:00 1.1.1970 stored as an integer in uint32 Note: capacity for approx. 136 years, i.e. until 2106	0x1234 5678	0	0x5678
					1	0x1234

Tab. 4 Real time

3.2.3 Real numbers

Type	Length	Range	Description	Notation in registers		
				Symbolic hex notation	Register offset	Register content
float	32 bit		IEEE 754 Single precision 32-bit, bit 31 sign (1 = negative number) bit 30 … 23 exponent 8bit bit 22 … 0 significand 24bit, the highest is not stated, always 1 Value = $(-1)^{sign}$ x $\left(1+\sum_{i=0}^{22} bit_{22-i} 2^{-1-i}\right)\left(2^{exponent-127}\right)$ Special numbers: 0 = 0x0000 0000 -0 = 0x8000 0000 NaN = exponent 0xFF, non-zero significand, sign has no effect ∞ = 0x7F80 0000 - ∞ = 0xFF80 0000	0x1234 5678	0	0x5678
					1	0x1234

Tab. 5 Basic real data types

3.2.4 Text strings

ASCII encoding is the primary encoding of text strings. Unless specified otherwise in the documentation, this encoding is used.

Type	Length	Description	Notation in registers (sample text "VF, a.s.")			
			Hex notation	Register offset	Register content	
					Character	Code
string	field 8 bit	text string with ASCII code in the range of 0x00, 0x20 … 0x7E, an even number of characters is always allocated, the string is ended with the first character with the code 0x00 or with the end of the defined string length	0x56 0x46 0x2C 0x20 0x61 0x2E 0x73 0x2E	0	'F' 'V'	0x4656
				1	, , ,,'	0x202C
				2	,.' ,a'	0x2E61
				3	,.' ,s'	0x2E73

Tab. 6 Text strings

Dec	Hex	Character	Dec	Hex	Character	Dec	Hex	Character	Dec	Hex	Character
0	0x00	NULL	55	0x37	7	79	0x4F	O	103	0x67	g
32	0x20	SPACE	56	0x38	8	80	0x50	P	104	0x68	h
33	0x21	!	57	0x39	9	81	0x51	Q	105	0x69	i
34	0x22	"	58	0x3A	:	82	0x52	R	106	0x6A	j
35	0x23	#	59	0x3B	;	83	0x53	S	107	0x6B	k
36	0x24	$	60	0x3C	<	84	0x54	T	108	0x6C	l
37	0x25	%	61	0x3D	=	85	0x55	U	109	0x6D	m
38	0x26	&	62	0x3E	>	86	0x56	V	110	0x6E	n
39	0x27	'	63	0x3F	?	87	0x57	W	111	0x6F	o
40	0x28	(	64	0x40	@	88	0x58	X	112	0x70	p
41	0x29	)	65	0x41	A	89	0x59	Y	113	0x71	q
42	0x2A	*	66	0x42	B	90	0x5A	Z	114	0x72	r
43	0x2B	+	67	0x43	C	91	0x5B	[	115	0x73	s
44	0x2C	,	68	0x44	D	92	0x5C	\	116	0x74	t
45	0x2D	-	69	0x45	E	93	0x5D	]	117	0x75	u
46	0x2E	.	70	0x46	F	94	0x5E	^	118	0x76	v
47	0x2F	/	71	0x47	G	95	0x5F	_	119	0x77	w
48	0x30	0	72	0x48	H	96	0x60	`	120	0x78	x
49	0x31	1	73	0x49	I	97	0x61	a	121	0x79	y
50	0x32	2	74	0x4A	J	98	0x62	b	122	0x7A	z
51	0x33	3	75	0x4B	K	99	0x63	c	123	0x7B	{
52	0x34	4	76	0x4C	L	100	0x64	d	124	0x7C	|
53	0x35	5	77	0x4D	M	101	0x65	e	125	0x7D	}
54	0x36	6	78	0x4E	N	102	0x66	f	126	0x7E	~

Tab. 7 Codes of ASCII characters

Recommended: When displaying ASCII strings the instrument / programme always checks the displayability of every single character. A non-displayable character is replaced with a substitute character, for example a rectangle with the usual size of one character.

3.3 Expanded data types

3.3.1 Integers

Type	Length	Range	Description	Notation in registers		
				Symbolic hex notation	Register offset	Register content
uint64	64 bit	0 … (2^{64} – 1)	unsigned long long	0x1234 5678 9ABC DEFG	0	0xDEFG
					1	0x9ABC
					2	0x5678
					3	0x1234
int164	64 bit	-2^{63} … (2^{63}-1)	signed long long, binary supplement, numbers -2^{63}… -1 conforming 0x8000 0000 0000 0000 … 0xFFFF FFFF FFFF FFFF, numbers 0 … (2^{31}-1) conforming 0x0000 0000 0000 0000 … 0x7FFF FFFF FFFF FFFF	0x1234 5678 9ABC DEFG	0	0xDEFG
					1	0x9ABC
					2	0x5678
					3	0x1234

Tab. 8 Expanded integer data types

3.3.2 Real numbers

Type	Length	Range	Description	Notation in registers		
				Symbolic hex notation	Register offset	Register content
double	64 bit		IEEE 754 Double precision 64-bit bit 63 sign (1 = negative number) bit 62 … 52 exponent bit 51 … 0 significand value = $(-1)^{sign}$ x $\left(1+\sum_{i=0}^{51} bit_{51-i}\, 2^{-1-i}\right)\left(2^{exponent-1023}\right)$ Special numbers: 0 = 0x0000 0000 0000 0000 -0 = 0x8000 0000 0000 0000 NaN = exponent 0x3FF, non-zero significand, sign has no effect 0xFFFF FFFF FFFF FFFF ∞ = 0x7FF0 0000 0000 0000 - ∞ = 0xFFF0 0000 0000 0000	0x1234 5678 9ABC DEFG	0	0xDEFG
					1	0x9ABC
					2	0x5678
					3	0x1234

Tab. 9 Expanded real data types

3.3.3 UTF-8 text strings

UTF-8 encoding is defined in [1] and related documents. It defines a space-saving method of Unicode character encoding. The first 127 UTF-8 characters are identical with ASCII. Characters beyond ASCII are encoded using two to four bytes. The first byte determines the number of bytes for one character. 2 bytes are usually enough for characters commonly used in Central Europe.

Range of Unicode characters		Bits	UTF-8 encoding			
from	to		Byte 0	Byte 1	Byte 2	Byte 3
0x0000 0000	0x0000 007F	7	0xxxxxxxb	-	-	-
0x0000 0080	0x0000 07FF	11	110xxxxxb	10xxxxxxb	-	-
0x0000 0800	0x0000 FFFF	16	1110xxxxb	10xxxxxxb	10xxxxxxb	-
0x0001 0000	0x0010 FFFF	21	11110xxxb	10xxxxxxb	10xxxxxxb	10xxxxxxb

Tab. 10 UTF-8 encoding

The organization of text in registers is almost identical with the basic string defined in section 3.2.4, with the only difference being the length of the text string. For ASCII encoding, the maximum number of characters corresponds directly to the length of the allocated field (number of registers * 2). For UTF-8 encoding beyond the framework of ASCII, the maximum number of characters depends on the length of their code and is always smaller than the length of the allocated field.

Type	Length	Description	Notation in registers (sample text "kódy")			
			Hex notation	Register offset	Register content	
					Character	Code
string	field 8 bit	text string with UTF-8 code allocated always an even number of bytes, the string is ended with the first character with the code 0x00 or with the end of the defined string length	0x6B 0xC3 0xB3 0x64 0x79 0x00	0	…ó' ,k'	0xC36B
				1	,d, ,ó…	0x64B3
				2	NULL ,y'	0x0079

Tab. 11 Text strings with UTF-8 characters

UTF-8	Character	Unicode		UTF-8	Character	Unicode	
		Dec	Hex			Dec	Hex
0xC381	Á	193	0x00C1	0xC3A1	á	225	0x00E1
0xC384	Ä	196	0x00C4	0xC3A4	ä	228	0x00E4
0xC48C	Č	268	0x010C	0xC48D	č	269	0x010D
0xC48E	Ď	270	0x010E	0xC48F	ď	271	0x010F
0xC389	É	201	0x00C9	0xC3A9	é	233	0x00E9
0xC49A	Ě	282	0x011A	0xC49B	ě	283	0x011B
0xC38D	Í	205	0x00CD	0xC3AD	í	237	0x00ED
0xC4B9	Ĺ	313	0x0139	0xC4BA	ĺ	314	0x013A
0xC4BD	Ľ	317	0x013D	0xC4BE	ľ	318	0x013E
0xC587	Ň	327	0x0147	0xC588	ň	328	0x0148
0xC393	Ó	211	0x00D3	0xC3B3	ó	243	0x00F3
0xC394	Ô	212	0x00D4	0xC3B4	ô	244	0x00F4
0xC594	Ŕ	340	0x0154	0xC595	ŕ	341	0x0155
0xC598	Ř	344	0x0158	0xC599	ř	345	0x0159
0xC5A0	Š	352	0x0160	0xC5A1	š	353	0x0161
0xC5A4	Ť	356	0x0164	0xC5A5	ť	357	0x0165
0xC39A	Ú	218	0x00DA	0xC3BA	ú	250	0x00FA
0xC5AE	Ů	366	0x016E	0xC5AF	ů	367	0x016F
0xC39D	Ý	221	0x00DD	0xC3BD	ý	253	0x00FD
0xC5BD	Ž	381	0x017D	0xC5BE	ž	382	0x017E
0xCEB1	α	945	0x03B1				
0xCEB2	β	946	0x03B2				
0xCEB3	γ	947	0x03B3				

Tab. 12 Selected UTF-8 character codes

3.4 Instrument identification

Start address	Number of registers	ID	Type	Description
0x0000	0x0002	HW_ID	uint32	HW identification, for encoding, see the table below. The designation of the closest semi-finished product characterising the specific connection point, in terms of communication, is always used. This refers to the module nomenclature in separate modules. If, from the point of view of communication, the specific module represents a higher unit in which it is incorporated (it is not a part replaceable by the user), it refers to the nomenclature of that higher unit.
0x0002	0x0002	SW_ID	uint32	SW/FW identification, for coding, see the table below.
0x0004	0x0001	SW_VER	uint16	"aa.bb" SW/FW versions coded as follows: (aa<< 8) | (bb) Example: 02.11 => 0x020B
0x0005	0x0001	-	-	Reserved for future additions.
0x0006	0x0002	SER_NO	uint32	The serial number of the instrument "xxxxxxx" It is used if, from the point of view of communication, the specific module represents a product or a top-end semi-finished product as a whole.
0x0008	0x0008	PR_ID	string	Variant "Kxxxx-xx" or nomenclature of the top-end semi-finished product, max. 16 characters. It is used if, from the point of view of communication, the specific module represents a product or a top-end semi-finished product as a whole. Example 1: The PCB module of the display represents the whole product PAM-100C, PR_ID contains the variant "K0952-01" Example 2: The PCB 1 PMT module of the detector represents the top-end whole semi-finished product SCD-286C, PR_ID contains the nomenclature of the semi-finished product "50-A-0000081"
0x0010	0x0008	USR_NAME	-	Optional user-defined device label, up to 16 characters. Defined according to project requirements.
0x0018	0x0008	PR_NAME	string	Product designation, max. 16 characters. It is used if, from the point of view of communication, the specific module represents a whole product.
0x0020	0x0002	PL_ID	uint32	Identification of FW programmable logic (CPLD/FPGA), for coding, see the table below.
0x0022	0x0001	PL_VER	uint16	Version of FW programmable logic (CPLD/FPGA) "aa.bb" coded as follows: (aa<< 8) | (bb) Example: 02.11 => 0x020B
0x0023	0x0001	-	-	Reserved for future additions.

Tab. 13 DEV_ID structure - instrument identification, RO area, length 0x0024

The existence of all device identification registers is mandatory. If a specific item is not used by a particular device, all its registers will return zero when read.

The explicit definition of the mandatory existence of registers from 0x0005 to 0x0023 was introduced in Revision B of this document.

The parameter USR_NAME was added, along with the corresponding main command, in Revision B of this document.

The content of SW_ID, SW_VER, PL_ID and PL_VER is only defined during project compilation. It is encoded into a uint32 type number, where the four upper bits define the type of the identifier, and the remaining bits contain the numerical value from the identifier without prefixes and separators.

Identifier based on SJZ / ERP	HW_ID	SW_ID PL_ID	Four upper bits	Example
Kxxxx-xx-Cxx	x		0x0[1]	K1234-56-C78 => 0x00BC614E
Kxxxx-xx-Fxx		x	0x0[1]	K1234-56-F78 => 0x00BC614E
50-A-xxxxxxx	x		0x1	50-A-1234567 => 0x1012D687
51-A-xxxxxxx	x		0x2	51-A-1234567 => 0x2012D687
52-A-xxxxxxx	x		0x3	52-A-1234567 => 0x3012D687
53-A-xxxxxxx		x	0x4	53-A-1234567 => 0x4012D687
54-A-xxxxxxx		x	0x5	54-A-1234567 => 0x5012D687
Sxxxx-xx-Fxx		x	0x8	S1234-56-F78 => 0x80BC614E
Reserved			not specified above	

Tab. 14 Encoding of identifiers HW_ID, SW_ID, PL_ID

3.5 Main command, entry data buffer

The main command is used for basic commands for the instrument. It involves the codes of functions. They are placed in a defined area in the address space.

Main Command Input Buffer (MCBUF) is used for some commands. It is a union where the content data type determines the main command used. It is usually identical to the data type of the written value.

The list of main commands and their parameters is provided in the documentation of the specific device.

[1] Attention, the value ID 0x00000000 cannot be presented as K0000-00-C00 or K0000-00-F00. If ID is zero for whichever reason (HW_ID does not have to be defined in the whole manufacturing process, PL_ID does not have to exist for the given product at all), it is not defined.

4 Description of the VFBUS protocol

4.1 VFBUS transactions

Data exchange takes place in two basic ways. Data are transmitted directly in one message without a request, or a request message is sent by the client and the server returns its response message (except for broadcast with the address 00h).

Data transmitted without a request cannot be combined within a single transaction with conventional queries and responses. Within one transaction, the instrument cannot be both a server and a client at the same time (its own request cannot be added to the response).

Limitation: Direct data transmission cannot be used in slave instruments on semi-duplex serial lines (RS-485)

The first byte of the message contains a transaction identifier. It serves for pairing requests with responses on the client side. The identifier determines the data source for directly transmitted data using the function 000b. For communication with a request, the identifier is determined by the client and the server copies it in the response.

In one VF PDU, up to 16 registers can be transmitted, i.e. 32 Bytes of data. The PDU mode occupies up to 3 Bytes. The maximum length of the message is 281 Bytes. It enables, when using eight PDUs, to transmit up to 128 registers (1 + 8 x (3 + 2x16) = 281 Bytes). Therefore, one transaction can transmit even the maximum volume of data from a single MODBUS PDU.

It is an obligation of the transaction creator to avoid both exceeding the maximum transaction length and requesting data that would exceed the maximum length in the server's response. When processing every single PDU, the server checks whether it will have space for a response. If not, the PDU is not processed, remaining PDUs, if any, are ignored in the transaction and the transaction is sent unfinished.

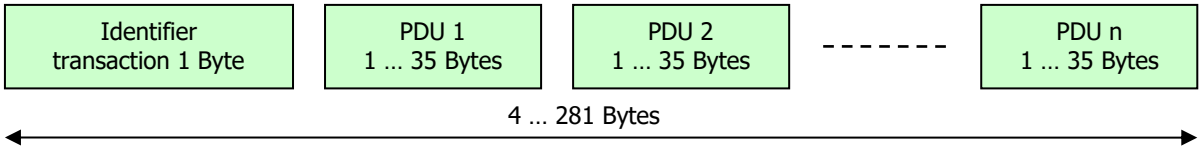

Fig. 3 Message structure

Offset	Length	Bit	Field	Description
0	1 Byte	7…5	Position	000b Direct data transmission 001b Reading request 010b Read data 011b Read error (non-existent register) 101b Write request 110b Write confirmation 111b Write error (non-existent register, RO, locked) 100b – reserved
		4	RO/RW space	0 – RO data 1 – RW data
		3 …0	Number of registers	0 … 15 number of transmitted or requested registers – 1
1	2 Bytes		First register address	0 … 0xFFFF
3	0 … 32 Bytes		Data	Only complete registers are transmitted, the data length is always even. Data are in the little endian format. The register with the lowest address goes first. Within one register, the lower byte goes first.

Note: All parts may not be always used, see the description of individual functions.

Tab. 15 Structure of VF PDU

PDU type Length	PDU item	Item length	Value	Description
Direct data transmission 3 +2*N Bytes	Position	3 bit	000b	Direct data transmission
	RO/RW space	1 bit	0 … 1b	RO space 0b RW space 1b
	Number of registers N-1	4 bit	0000b … 1111b	Number of transmitted registers N – 1
	First register address	2 Bytes	0 … 0xFFFF	RO/RW address of the data source space
	Register values	N x 2 Bytes	N x 0 … 0xFFFF	Transmitted data

Tab. 16 Direct data transmission

PDU type Length	PDU item	Item length	Value	Description
Request Reading 3 Bytes	Position	3 bit	001b	Read request
	RO/RW space	1 bit	0 … 1b	RO space 0b RW space 1b
	Number of registers N-1	4 bit	0000b … 1111b	Number of requested registers N - 1
	First register address	2 Bytes	0 … 0xFFFF	RO/RW address in the server space
Read data 3 +2*N Bytes	Position	3 bit	010b	Read data. Only when all the requested registers can be transmitted.
	RO/RW space	1 bit	0 … 1b	RO space 0b RW space 1b
	Number of registers N-1	4 bit	0000b … 1111b	Number of transmitted registers N – 1
	First register address	2 Bytes	0 … 0xFFFF	RO/RW address in the server space
	Register values	2*N Bytes	N* 0 … 0xFFFF	Read data
Read error 1 Byte	Position	3 bit	011b	Read error. If at least one non-existent register is requested, no data are transmitted.
	RO/RW space	1 bit	0 … 1b	RO space 0b RW space 1b
	Not used	4 bit	0000b	Not used, always 0

Tab. 17 Read procedure

PDU type Length	PDU item	Item length	Value	Description
Write request 3 + 2*N Bytes	Position	3 bit	101b	Write request
	RO/RW space	1 bit	1b	Always RW space
	Number of registers N-1	4 bit	0x0 … 0xF	Number of written registers N – 1
	First register address	2 Bytes	0 … 0xFFFF	RO/RW address in the server space
	Register values	2*N Bytes	N * 0 … 0xFFFF	Written data
Write confirmation 1 Byte	Position	3 bit	110b	Write confirmation, if all requested registers have been written.
	RO/RW space	1 bit	1b	Always RW space
	Not used	4 bit	0000b	Not used, always 0
Write error 1 Byte	Position	3 bit	111b	Write error. If writing is not possible in at least one of the requested registers, nothing is written.
	RO/RW space	1 bit	0 … 1b	RO space 0b RW space 1b
	Not used	4 bit	0000b	Not used, always 0

Tab. 18 Write procedure

The command register must be processed on the server side immediately **after receiving each individual PDU.**

The handling method must be chosen based on the nature of the specific command. It may involve immediate execution (e.g., item selector for reading, time update command, etc.), or registration of the request for later processing (e.g., device reset, change of alarm level, etc.).

The explicit requirement to process the PDU upon reception was introduced in revision B of this document. Some older devices do not comply with this requirement, which leads to the need for a time delay between command register writes.

4.2 Addressing

A communicating system always includes at least two stations (devices, modules, computers, etc.). However, in many cases, there are more stations involved, forming a communication network. The network can grow horizontally, by increasing the number of stations on a single line, and/or vertically, where some stations interconnect two or more lines. If a station has multiple lines (ports), it often also has multiple communication addresses. An address is always unique on a single line, but uniqueness across the entire system is not required.

In VFBUS protocols, both a recipient address and a sender address are used. This addressing pair enables multi-master communication and message routing. The recipient address defines the target station(s) for a message. The sender address is used to route the response back to the original sender. During transaction processing, the recipient becomes the sender and vice versa. A range of standard addresses is defined in accordance with MODBUS protocol addressing (defined in Chapter 4.3), along with several special addresses. An overview is provided in Table 28.

In Ethernet-based interfaces, station addressing operates on two levels. The upper level is the Ethernet network (IP address, port, etc.), while the lower level consists of sender and recipient addresses (identifiers) used within the VFBUS protocols.

Address	Value	Description
Recipient	0	Broadcast without response. Each recipient processes the transaction but does not send any response. Used when data needs to be delivered to multiple recipients simultaneously (e.g., time synchronization).
Recipient	1-247	Recipient address on the given port. The recipient with the specified address processes the transaction and, if it is a request, sends a response.
Recipient	255	Broadcast with response. Each recipient processes the transaction. If it is a request, each sends a response and includes its own address in the response. Used when the recipient's address is unknown (e.g., device discovery) or irrelevant (e.g., peer-to-peer communication on a full-duplex serial line). Protocol limitation: Broadcast is not supported in VFC-B.
Sender	0	Reserved.
Sender	1-247	Sender address on the given port. The recipient uses this address as the recipient address in its response.
Sender	255	Unspecified sender address – allowed only in VFBUS SL over duplex line and VFBUS UDP. Upon receiving a message with an unspecified sender address, two cases may occur: a) If the recipient does not expect a specific sender address (typical for end devices), it uses address 255 as the recipient address in the response, just like a standard address. b) If the recipient expects a specific sender address or a range of addresses (typical for routing devices), it replaces address 255 with the expected value before processing the request (i.e., creating a response or forwarding the message to another communication link).
Recipient and sender	248-254	Reserved.

Tab. 19 Addressing in devices

Addresses 248–254 are not used due to the inability to route messages between the VFBUS and MODBUS protocols.

The behaviour of a recipient when receiving a message with the unspecified sender address (255) was defined starting from Revision B of this document.

4.3 VFBUS SL protocol

The VFBUS SL (abbreviated as VFS) protocol is used on serial lines. The message starts with a start character followed by the recipient's address, transaction data, checksum, and the end character.

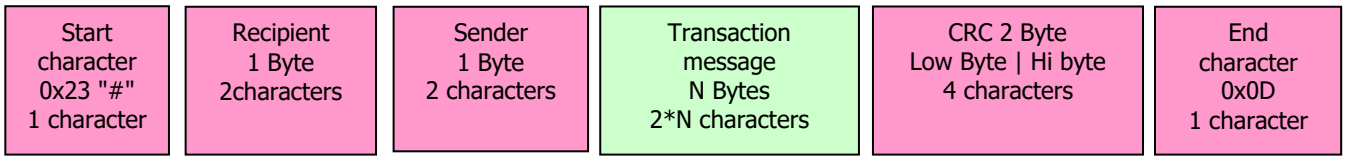

Fig. 4 VFBUS SL protocol message

The data is transmitted as text. All bytes between the direction identifier and the end character are coded with two ASCII characters in a hexadecimal format. Example: the value of byte 0x5B is coded with 0x35 ("5") and 0x42 ("B"). They are transmitted as they are found in the memory of the little-endian data space, i.e. the lowest byte goes first. The number of message characters is 1 + 2 + 2 + 8 up to 562 + 4 + 1, i.e. 572 characters at the maximum.

The checksum algorithm is identical with MODBUS RTU, see section 4.4. Calculation inputs are individual characters of the message. The start character is omitted; counting starts from the first character of the recipient's address and continues to the last character in the transaction.

The end instrument starts sending the response within **100 ms** of receipt of the last request character. In semi-duplex physical layers (RS485), the minimum time of response is equivalent to **3 characters** on the line. The minimum timeout from sending the last character of the request is 100ms + 572 characters x 10 bits / baud rate.

If the data passes through the gateway or is identified by the server in another station, the timeouts need to be selected regarding the entire line.

Encoding	**8 bit binary**
Individual bits	1 start bit
	8 data bits
	1 stop bit
Baud rate	obligatory: 9600 other: 4800, 9600, 19200, 38400, 57600, 115200 switching by SW/HW, start not defined by the standard

Tab. 20 Setting of an asynchronous serial line for the VFS protocol

4.4 Checksum calculation

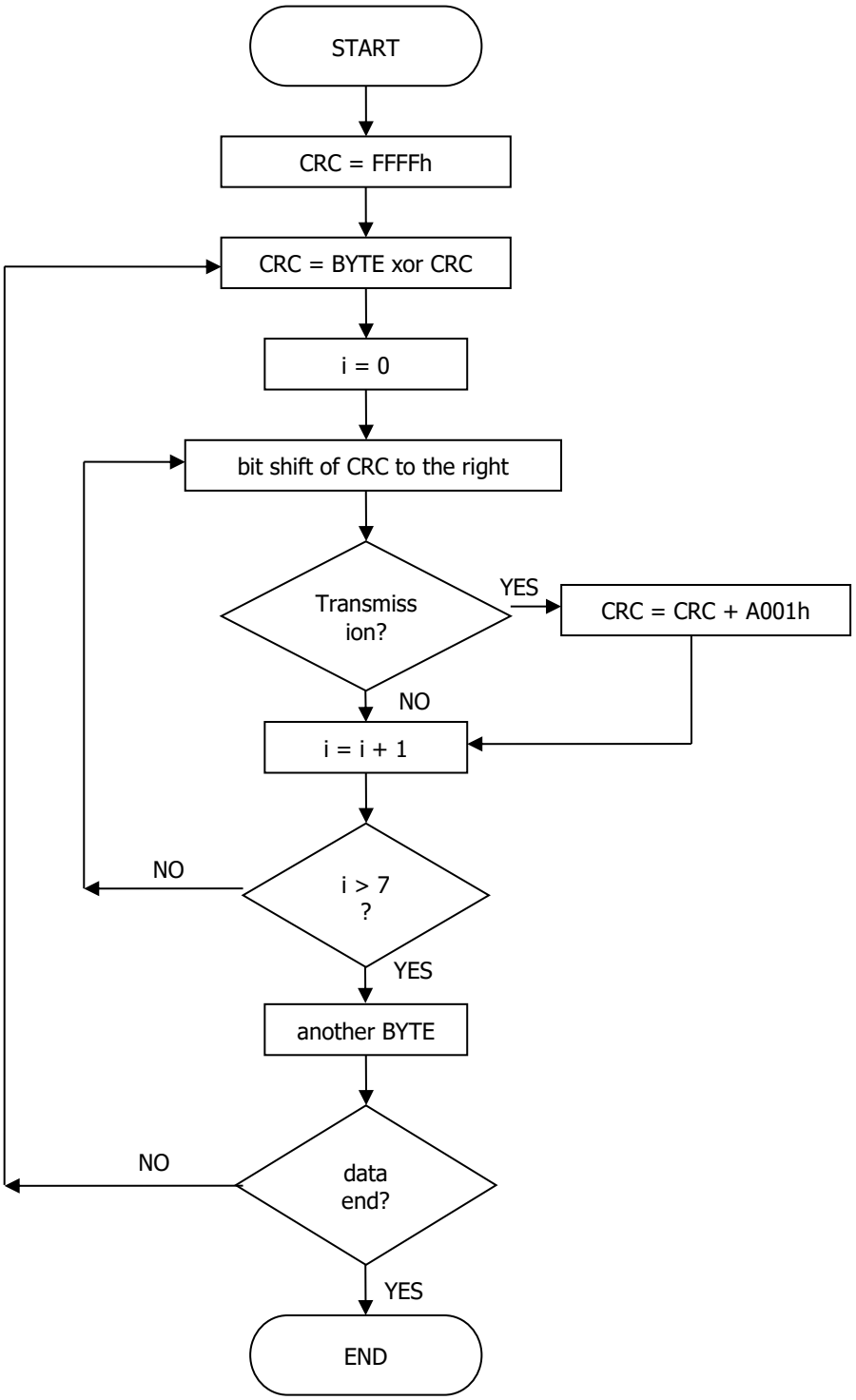

Fig. 5 CRC checksum calculation

The CRC checksum can also be calculated using a table. The calculation is more efficient.

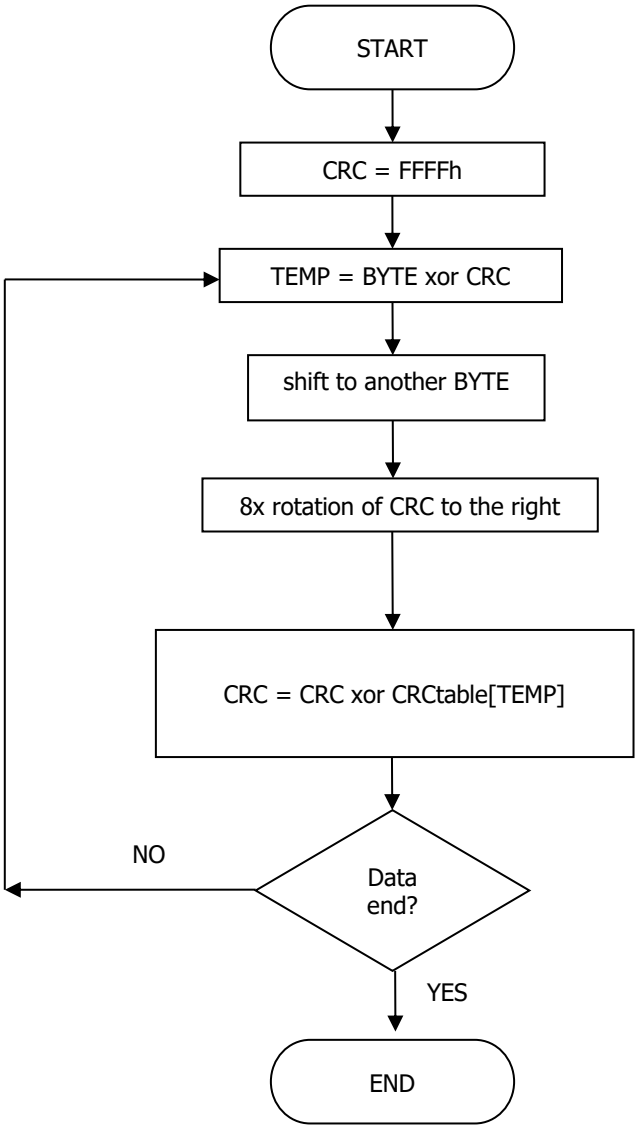

Fig. 6 CRC checksum calculation using a table

Example of CRC calculation using a table in the C language:

```
uint16 modbus_getCRC (uint8 *nData, uint16 len) {
uint8 Temp;
uint16 CRC = 0xFFFF;

  while (len--) {
    Temp = *nData++ ^ CRC;
    CRC >>= 8;
    CRC ^= CRCTable[Temp];
  }
  return CRC;
}

const uint16 CRCTable[] = {
0x0000, 0xC0C1, 0xC181, 0x0140, 0xC301, 0x03C0, 0x0280, 0xC241,
0xC601, 0x06C0, 0x0780, 0xC741, 0x0500, 0xC5C1, 0xC481, 0x0440,
0xCC01, 0x0CC0, 0x0D80, 0xCD41, 0x0F00, 0xCFC1, 0xCE81, 0x0E40,
0x0A00, 0xCAC1, 0xCB81, 0x0B40, 0xC901, 0x09C0, 0x0880, 0xC841,
0xD801, 0x18C0, 0x1980, 0xD941, 0x1B00, 0xDBC1, 0xDA81, 0x1A40,
0x1E00, 0xDEC1, 0xDF81, 0x1F40, 0xDD01, 0x1DC0, 0x1C80, 0xDC41,
0x1400, 0xD4C1, 0xD581, 0x1540, 0xD701, 0x17C0, 0x1680, 0xD641,
0xD201, 0x12C0, 0x1380, 0xD341, 0x1100, 0xD1C1, 0xD081, 0x1040,
0xF001, 0x30C0, 0x3180, 0xF141, 0x3300, 0xF3C1, 0xF281, 0x3240,
0x3600, 0xF6C1, 0xF781, 0x3740, 0xF501, 0x35C0, 0x3480, 0xF441,
0x3C00, 0xFCC1, 0xFD81, 0x3D40, 0xFF01, 0x3FC0, 0x3E80, 0xFE41,
0xFA01, 0x3AC0, 0x3B80, 0xFB41, 0x3900, 0xF9C1, 0xF881, 0x3840,
0x2800, 0xE8C1, 0xE981, 0x2940, 0xEB01, 0x2BC0, 0x2A80, 0xEA41,
0xEE01, 0x2EC0, 0x2F80, 0xEF41, 0x2D00, 0xEDC1, 0xEC81, 0x2C40,
0xE401, 0x24C0, 0x2580, 0xE541, 0x2700, 0xE7C1, 0xE681, 0x2640,
0x2200, 0xE2C1, 0xE381, 0x2340, 0xE101, 0x21C0, 0x2080, 0xE041,
0xA001, 0x60C0, 0x6180, 0xA141, 0x6300, 0xA3C1, 0xA281, 0x6240,
0x6600, 0xA6C1, 0xA781, 0x6740, 0xA501, 0x65C0, 0x6480, 0xA441,
0x6C00, 0xACC1, 0xAD81, 0x6D40, 0xAF01, 0x6FC0, 0x6E80, 0xAE41,
0xAA01, 0x6AC0, 0x6B80, 0xAB41, 0x6900, 0xA9C1, 0xA881, 0x6840,
0x7800, 0xB8C1, 0xB981, 0x7940, 0xBB01, 0x7BC0, 0x7A80, 0xBA41,
0xBE01, 0x7EC0, 0x7F80, 0xBF41, 0x7D00, 0xBDC1, 0xBC81, 0x7C40,
0xB401, 0x74C0, 0x7580, 0xB541, 0x7700, 0xB7C1, 0xB681, 0x7640,
0x7200, 0xB2C1, 0xB381, 0x7340, 0xB101, 0x71C0, 0x7080, 0xB041,
0x5000, 0x90C1, 0x9181, 0x5140, 0x9301, 0x53C0, 0x5280, 0x9241,
0x9601, 0x56C0, 0x5780, 0x9741, 0x5500, 0x95C1, 0x9481, 0x5440,
0x9C01, 0x5CC0, 0x5D80, 0x9D41, 0x5F00, 0x9FC1, 0x9E81, 0x5E40,
0x5A00, 0x9AC1, 0x9B81, 0x5B40, 0x9901, 0x59C0, 0x5880, 0x9841,
0x8801, 0x48C0, 0x4980, 0x8941, 0x4B00, 0x8BC1, 0x8A81, 0x4A40,
0x4E00, 0x8EC1, 0x8F81, 0x4F40, 0x8D01, 0x4DC0, 0x4C80, 0x8C41,
0x4400, 0x84C1, 0x8581, 0x4540, 0x8701, 0x47C0, 0x4680, 0x8641,
0x8201, 0x42C0, 0x4380, 0x8341, 0x4100, 0x81C1, 0x8081, 0x4040};
```

5 Example of communication

Task: Instrument A reads 30 registers from the RW area of instrument B, from address 0x0AB5

The register values in instrument B from address 0x0AB5 in the RW area are:

0x0100 0x0302 0x0504 0x0706 0x0908 0x0B0A 0x0D0C 0x0F0E
0x1110 0x1312 0x1514 0x1716 0x1918 0x1B1A 0x1D1C 0x1F1E
0x2120 0x2322 0x2524 0x2726 0x2928 0x2B2A 0x2D2C 0x2F2E
0x3130 0x3332 0x3534 0x3736 0x3938 0x3B3A

Number of transaction in instrument A [1]	0xFE
Identifier (address) of instrument A [2]	0x08
Identifier (address) of instrument B	0x17

5.1 VFBUS transactions

30 registers of the RW area can be transmitted with a single transaction, it is necessary to use two PDUs for reading. The first PDU reads 16 registers, the other PDU reads 14 registers.

Request message:

[ID][PDU1][PDU2]

[0xFE][0x3FB50A][0x3DC50A]

Response message:

[ID][PDU1][PDU2]

[0xFE][0x5FB50A000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F][0x5DC50A20212223242526272829 2A2B2C2D2E2F303132333435363738393A3B]

5.1.1 VFBUS SL protocol

Request packet (text string):

[start character][recipient][sender][transaction message][CRC][end character]

[#][17][08][FE3FB50A3DC50A][A148][<0x0D>]

Response packet (text string):

[start character][recipient][sender][transaction message][CRC][end character]

[#][08][17][FE5FB50A000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F5DC50A2021 22232425262728292A2B2C2D2E2F303132333435363738393A3B][F4E0][<0x0D>]

[1] RFC 3629 UTF-8, a transformation format of ISO 10646, http://www.rfc-editor.org

[1] Not used in MODBUS RTU
[2] Not used in MODBUS RTU and MODBUS on TCP/IP

6 Documents Used

[1] RFC 3629 UTF-8, a transformation format of ISO 10646, http://www.rfc-editor.org

7 Annexes

N/A

8 Final provisions

On the effective date of this VF Standard, the previous version of the document is cancelled.